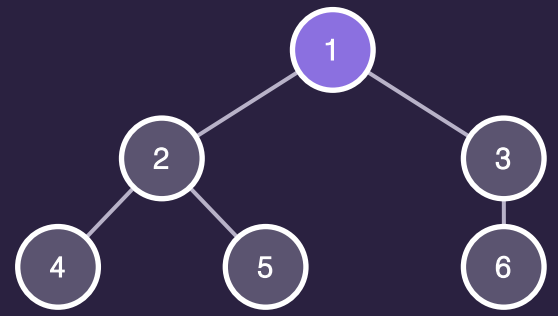


파이썬으로 배우는 DFS와 BFS

그래프 탐색 — 깊이 우선 탐색과 너비 우선 탐색

CodeWithCathy

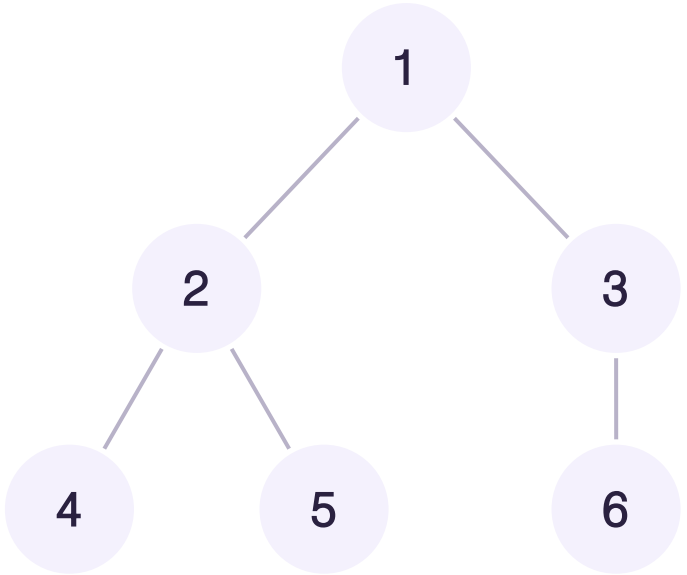
그래프란?

점(노드)과 점을 잇는 선(간선)으로 관계를 표현한 것

친구 관계, 지하철 노선, 미로의 칸처럼 “무엇이 무엇과 연결돼 있는지”를 나타내는 건 모두 그래프로 볼 수 있어요.

그래프 탐색이란

연결된 노드를 빠짐없이 방문하기

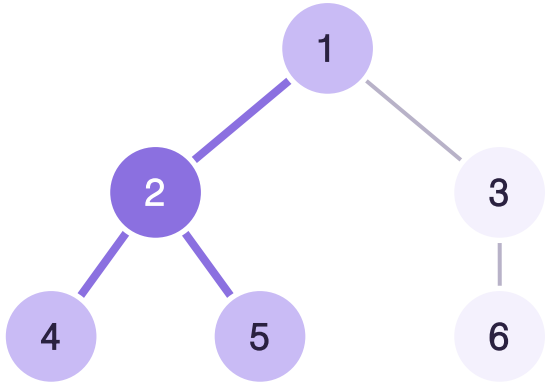


● 노드 1, 2, 3, 4, 5, 6
— 간선 1–2, 1–3, 2–4, 2–5, 3–6

파이썬으로 그래프 표현하기: 인접 리스트

Python

```python
graph = {
    1: [2, 3],
    2: [1, 4, 5],
    3: [1, 6],
    4: [2],
    5: [2],
    6: [3],
}
```



노드마다 “나와 연결된 노드 목록”을 저장해요

`graph[2]` → `[1, 4, 5]` : 2번에서 갈 수 있는 곳

양방향이면 두 번 적기

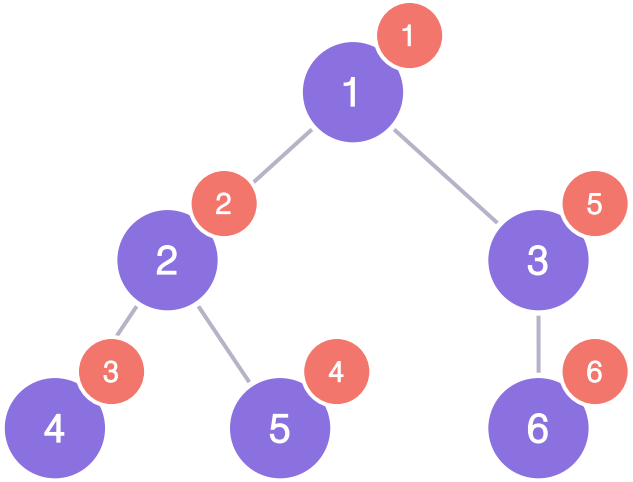
1–2가 연결이면 graph[1]에 2, graph[2]에 1을 둘 다 넣어요

간선 목록으로 주어지면 `for a, b in edges:` 로 돌면서 양쪽에 append

DFS와 BFS, 무엇이 다를까?

DFS 깊이 우선 탐색

한 길을 끝까지 가본 뒤 돌아와서 다른 길로

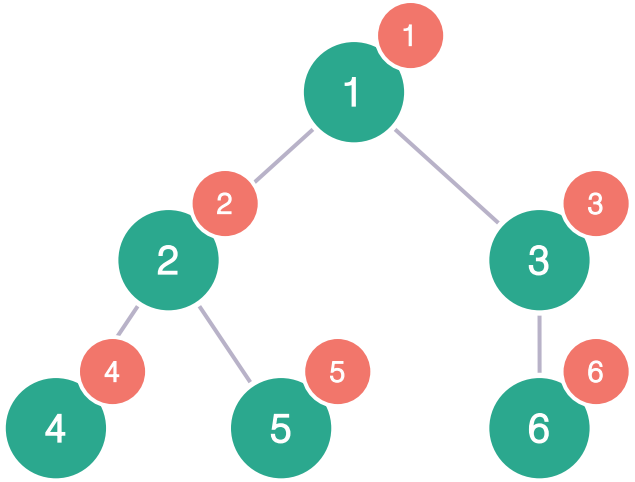


방문 순서 **1 → 2 → 4 → 5 → 3 → 6**

사용 도구 **스택 / 재귀**

BFS 너비 우선 탐색

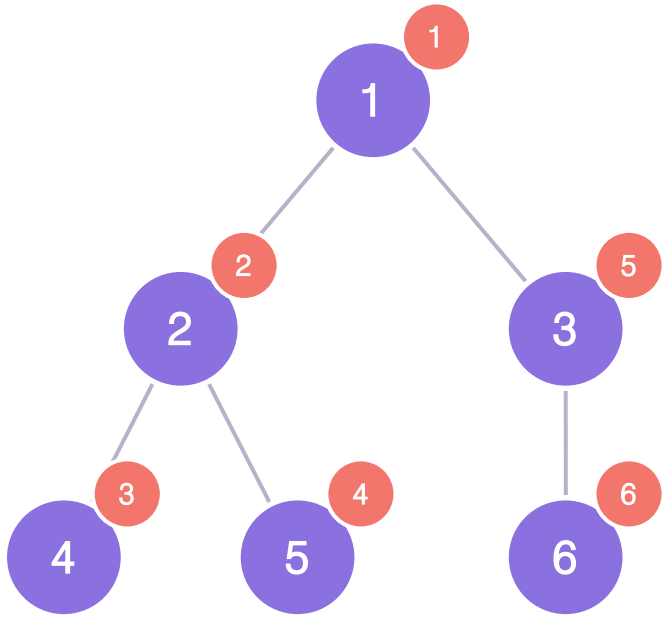
가까운 노드부터 한 겹씩 넓혀가며



방문 순서 **1 → 2 → 3 → 4 → 5 → 6**

사용 도구 **큐 (deque)**

DFS 따라가 보기



숫자 = 방문 순서

단계	지금 노드	들어온 길 (스택)	방문한 노드
1	1	[1]	1
2	2	[1, 2]	1 2
3	4	[1, 2, 4]	1 2 4
↩	2로 돌아감	[1, 2]	4는 막다른 길
4	5	[1, 2, 5]	1 2 4 5
↩	1로 돌아감	[1]	2의 이웃 다 봄
5	3	[1, 3]	1 2 4 5 3
6	6	[1, 3, 6]	1 2 4 5 3 6

핵심 더 갈 곳이 없으면 가장 최근에 지나온 곳으로 되돌아가요 → 스택(LIFO)

DFS 코드: 재귀와 스택 두 가지

재귀

```python
visited = []

def dfs(node):
    visited.append(node)
    for nxt in graph[node]:
        if nxt not in visited:
            dfs(nxt)

dfs(1)   # [1, 2, 4, 5, 3, 6]
```

스택

```python
def dfs(start):
    visited = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.append(node)
        for nxt in reversed(graph[node]):
            stack.append(nxt)
    return visited
```

1

재귀가 더 짧아요

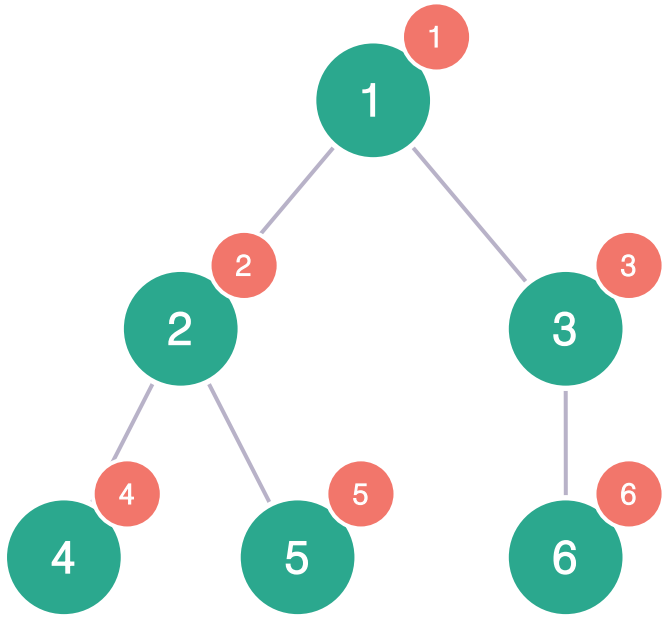
함수 호출 자체가 스택처럼 쌓였다가 돌아와요

!

reversed를 쓰는 이유

스택은 나중에 넣은 걸 먼저 꺼내서, 작은 번호부터 가려면 거꾸로 넣어요

BFS 따라가 보기



1겹: 1 → 2겹: 2, 3 → 3겹: 4, 5, 6

단계	꺼낸 노드	큐에 넣은 노드	큐 상태 (앞 ← 뒤)
시작	-	1	[1]
1	1	2, 3	[2, 3]
2	2	4, 5	[3, 4, 5]
3	3	6	[4, 5, 6]
4	4	-	[5, 6]
5	5	-	[6]
6	6	-	[] 끝

핵심 먼저 발견한 노드부터 꺼내요 → 큐(FIFO) → 가까운 노드부터 방문

BFS 코드: deque로 구현하기

Python

```python
from collections import deque

def bfs(start):
    visited = [start]
    q = deque([start])
    while q:
        node = q.popleft()
        for nxt in graph[node]:
            if nxt not in visited:
                visited.append(nxt)
                q.append(nxt)
    return visited   # [1, 2, 3, 4, 5, 6]
```

1 **시작점을 넣고 방문 표시**

visited와 큐에 start를 함께 넣어요

2 **큐 앞에서 하나 꺼내기**

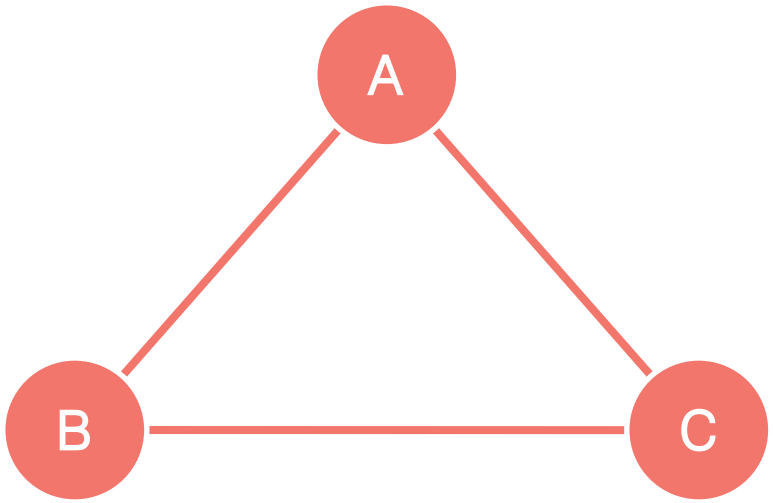
popleft()로 가장 먼저 들어온 노드

3 **안 간 이웃은 표시 후 큐 뒤에**

넣을 때 바로 방문 표시해야 중복이 안 생겨요

비교 DFS 스택 버전과 뼈대가 비슷해요. 스택(pop) 대신 큐(popleft)를 쓰는 게 핵심 차이예요

visited를 빼먹으면 어떻게 될까?



빙글빙글 무한 반복

그래프에 고리(사이클)가 있으면 왔던 곳을 또 가요. DFS는 RecursionError, BFS는 끝나지 않아요

방문 표시로 해결

한 번 간 노드는 visited에 기록하고, 다음엔 건너뛰어요

규칙 DFS든 BFS든 “방문 표시 → 안 간 곳만 가기”는 무조건 들어가요

2차원 격자도 그래프예요

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3

칸 = 노드, 상하좌우로 붙은 칸 = 간선

(행, 열)로 위치를 나타내고, 이웃은 인접 리스트 대신 방향 배열로 계산해요

Python

```python
# 아래, 위, 오른쪽, 왼쪽
dirs = [(1, 0), (-1, 0), (0, 1), (0, -1)]

for dr, dc in dirs:
    nr, nc = r + dr, c + dc
    # 지도 밖으로 나가지 않는지 확인
    if 0 <= nr < rows and 0 <= nc < cols:
        ...
```

범위 확인 먼저

지도 밖 칸을 보면 IndexError, 음수 인덱스는 반대편 칸

그래프 탐색 문제를 푸는 4단계

1

그래프 만들기

인접 리스트? 격자?

```
graph = {}
grid[r][c]
```

2

방문 표시 준비

무엇으로 기록할까?

```
visited = set()
[[False]*m for
 _ in range(n)]
```

3

DFS? BFS?

최단 거리면 BFS

```
최단 거리 → BFS
그 외 → 편한 쪽
```

4

탐색하며 답 구하기

개수? 거리? 경로?

```
count += 1
dist[nxt] = ...
```

이제 이 4단계로 간단한 예제 두 개를 풀어볼게요

예제 ① 섬의 개수 세기 (DFS)

문제 1은 땅, 0은 바다. 상하좌우로 붙은 땅을 하나의 섬으로 볼 때 섬은 몇 개?

1	1	0	0	1	0
1	0	0	1	1	0
0	0	1	0	0	0
0	1	1	0	1	1
0	0	0	0	1	0

색이 같은 칸 = 같은 섬 → 정답 4

1. **모든 칸을 차례로 확인**

   땅인데 아직 방문하지 않은 칸을 찾아요

2. **찾으면 DFS로 섬 전체 방문**

   붙어 있는 땅을 모두 방문 표시해요

3. **섬 개수 + 1**

   DFS를 새로 시작한 횟수 = 섬의 개수

예제 ①: 코드로 옮기기

Python

```python
def solution(grid):
    rows, cols = len(grid), len(grid[0])
    visited = [[False] * cols for _ in range(rows)]

    def dfs(r, c):
        visited[r][c] = True
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                if grid[nr][nc] == 1 and not visited[nr][nc]:
                    dfs(nr, nc)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1 and not visited[r][c]:
                dfs(r, c)   # 새 섬 발견
                count += 1
    return count
```

1

dfs를 안쪽에

solution 안에 두면 grid, visited를 바로 쓸 수 있어요

2

두 if로 나눈 이유

범위를 먼저 확인해야 grid[nr][nc]에서 에러가 안 나요

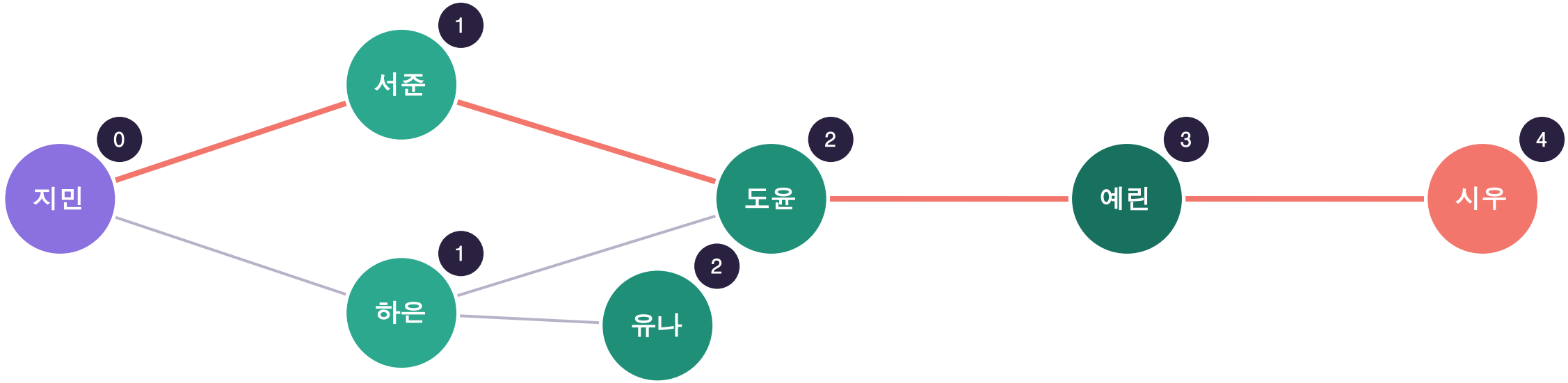
!

격자가 크면

재귀 깊이 제한에 걸릴 수 있어요

예제 ② 친구 몇 다리 건너 아는 사이? (BFS)

문제 친구 관계가 주어질 때, 지민에서 시우까지 최소 몇 다리를 건너야 할까?



작은 숫자 = 지민에서 몇 다리인지 (BFS가 한 겹씩 넓혀가며 기록)

지민–서준–도윤–하은이 고리를 이루니까 visited가 꼭 필요해요

정답 4

예제 ②: 코드로 옮기기

Python

```python
from collections import deque

def solution(friends, start, end):
    graph = {}
    for a, b in friends:
        graph.setdefault(a, []).append(b)
        graph.setdefault(b, []).append(a)

    dist = {start: 0}   # 방문 표시 겸 거리 기록
    q = deque([start])
    while q:
        now = q.popleft()
        if now == end:
            return dist[now]
        for nxt in graph[now]:
            if nxt not in dist:
                dist[nxt] = dist[now] + 1
                q.append(nxt)
    return -1   # 연결이 안 돼 있으면
```

1

setdefault

키가 없으면 빈 리스트를 만들고, 있으면 그대로 써요

2

dist 하나로 두 역할

dist에 있으면 방문한 것, 값은 몇 다리인지

✓

왜 BFS?

가까운 사람부터 보니까 처음 만나면 최단

DFS vs BFS 한눈에 비교

구분	DFS 깊이 우선	BFS 너비 우선
탐색 방식	한 길 끝까지 → 되돌아오기	가까운 곳부터 한 겹씩
자료구조	스택 / 재귀	큐 (deque)
최단 거리	**보장 안 됨**	**보장됨**
잘 맞는 문제	덩어리 세기, 모든 경로·조합	최단 거리, 최소 횟수
주의할 점	재귀 깊이 제한	큐에 넣을 때 방문 표시

고르는 기준 **“최소 / 최단”이 보이면 BFS, 나머지는 편한 쪽으로**

오늘의 정리

1

그래프 = 노드 + 간선, 파이썬에서는 인접 리스트

격자도 그래프: 칸이 노드, 상하좌우가 간선

2

DFS는 스택·재귀, BFS는 큐

DFS는 한 길 끝까지, BFS는 가까운 곳부터 한 겹씩

3

방문 표시는 필수, 최단 거리는 BFS

visited 없으면 무한 반복 · “최소/최단”이 보이면 BFS